

# **Video Processing using OpenCV – Functions and Mathematical Concepts**

## **1. cv2.VideoCapture()**

### **Purpose**

This function is used to **capture video from a webcam or video file**.

### **Syntax**

```
cap = cv2.VideoCapture(0)
```

or

```
cap = cv2.VideoCapture("edge_vid1.avi")
```

### **Mathematical Concept**

A video can be represented as a **sequence of frames**.

$$Video = \{F_1, F_2, F_3, \dots, F_n\}$$

Where

- $F_i$  =  $i$ th frame
  - Each frame is an image matrix.
- 

## **2. cap.isOpened()**

### **Purpose**

Checks whether the **camera or video file opened successfully**.

### **Syntax**

```
cap.isOpened()
```

### **Concept**

Returns a **Boolean value**

$$Result = \begin{cases} True & \text{camera detected} \\ False & \text{camera not detected} \end{cases}$$

---

### 3. cap.get()

#### Purpose

Retrieves **video properties** such as frame width, height, and FPS.

#### Syntax

```
frame_width = cap.get(3)
frame_height = cap.get(4)
```

Other properties:

- CAP\_PROP\_FRAME\_COUNT
- CAP\_PROP\_FPS
- CAP\_PROP\_FRAME\_WIDTH
- CAP\_PROP\_FRAME\_HEIGHT

#### Mathematical Representation

Frame size:

$$\text{Resolution} = \text{Width} \times \text{Height}$$

Example

$$640 \times 480$$

---

### 4. cv2.VideoWriter()

#### Purpose

Used to **save captured frames into a video file**.

#### Syntax

```
out = cv2.VideoWriter('edge_vid1.avi', codec, fps, (width,height))
```

#### Mathematical Concept

A video is created by combining frames:

$$Video = F_1 + F_2 + F_3 + \dots + F_n$$

---

## 5. cv2.VideoWriter\_fourcc()

### Purpose

Defines the **video compression codec**.

### Syntax

```
cv2.VideoWriter_fourcc(*'XVID')
```

### Concept

FOURCC = **Four Character Code**

Used to encode video format.

---

## 6. cap.read()

### Purpose

Reads frames from the camera or video.

### Syntax

```
ret, frame = cap.read()
```

### Mathematical Concept

Each frame is a **pixel matrix**

$$Frame = \begin{bmatrix} p_{11} & p_{12} & \dots \\ p_{21} & p_{22} & \dots \end{bmatrix}$$

---

## 7. cv2.cvtColor()

### Purpose

Converts image from **BGR to Grayscale**.

### Syntax

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

### **Mathematical Formula**

Grayscale conversion:

$$Gray = 0.299R + 0.587G + 0.114B$$

Where

- $R$ = red channel
  - $G$ = green channel
  - $B$ = blue channel
- 

## **8. cv2.GaussianBlur()**

### **Purpose**

Reduces image noise and smooths the image.

### **Syntax**

```
blur = cv2.GaussianBlur(gray, (5,5), 0)
```

### **Gaussian Function**

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This filter smooths pixel intensities.

---

## **9. cv2.Canny()**

### **Purpose**

Detects edges in an image.

### **Syntax**

```
edges = cv2.Canny(blur,100,200)
```

### **Gradient Magnitude**

$$G = \sqrt{G_x^2 + G_y^2}$$

Where

$$G_x = \frac{\partial I}{\partial x}$$

$$G_y = \frac{\partial I}{\partial y}$$

Edges occur where intensity changes rapidly.

---

## 10. cv2.imwrite()

### Purpose

Saves a frame as an image file.

### Syntax

```
cv2.imwrite("frame_100.jpg", frame)
```

### Concept

Stores pixel matrix as an image file.

---

## 11. cv2.imread()

### Purpose

Reads an image from disk.

### Syntax

```
img = cv2.imread("frame_100.jpg")
```

### Mathematical Representation

Image as matrix:

$$I(x, y)$$

For RGB image:

$$Pixel = (R, G, B)$$

---

## 12. cv2.rectangle()

### Purpose

Draws a rectangle on the image.

### Syntax

```
cv2.rectangle(img,(x1,y1),(x2,y2),(0,255,0),3)
```

### Mathematical Concept

Rectangle defined by two corner points:

$$Width = x_2 - x_1$$

$$Height = y_2 - y_1$$

---

## 13. cv2.circle()

### Purpose

Draws a circle on the image.

### Syntax

```
cv2.circle(img,(x,y),radius,(255,0,0),3)
```

### Circle Equation

$$(x - h)^2 + (y - k)^2 = r^2$$

Where

- $h, k$  = center
  - $r$  = radius
-

## 14. cv2.line()

### Purpose

Draws a straight line between two points.

### Syntax

```
cv2.line(img,(x1,y1),(x2,y2),(0,0,255),3)
```

### Line Equation

$$y = mx + c$$

Where

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

---

## 15. cv2.putText()

### Purpose

Writes text on the image.

### Syntax

```
cv2.putText(img,"Static Capture",(50,400),  
            cv2.FONT_HERSHEY_SIMPLEX,1,(255,255,255),2)
```

### Concept

Text is placed at coordinate

$$Position = (x, y)$$

---

## 16. cv2.imshow()

### Purpose

Displays an image or video frame.

### Syntax

```
cv2.imshow("Window Name", frame)
```

### **Concept**

Displays the **pixel matrix on screen**.

---

## **17. cv2.waitKey()**

### **Purpose**

Waits for keyboard input.

### **Syntax**

```
cv2.waitKey(1)
```

Example:

```
if cv2.waitKey(1) & 0xFF == ord('q'):
```

Press **q** to exit the loop.

---

## **18. Negative Transformation**

### **Code Used**

```
negative = 255 - frame
```

### **Mathematical Formula**

$$s = 255 - r$$

### **Where**

- **$r$** = original pixel value
- **$s$** = negative pixel value

**Dark pixels become bright and bright pixels become dark.**

---

## **19. Frame Difference (Motion Detection)**

Used when comparing two frames.



## Concept

$$D(x, y) = | F_1(x, y) - F_2(x, y) |$$

If difference is large  $\rightarrow$  motion detected.

---

## 20. cap.release()

### Purpose

Releases the camera resource.

cap.release()

---

## 21. cv2.destroyAllWindows()

### Purpose

Closes all OpenCV windows.

cv2.destroyAllWindows()

---



